

BitsAndBytesConfig

QLoRA ও 8-বিট কোয়ান্টাইজেশনের সম্পূর্ণ বাংলা গাইড



4-Bit Quantization



NF4 Format



bfloat16



QLoRA



Python

```
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.bfloat16
)
```



বিষয়বস্তু সূচি

১

BitsAndBytesConfig কী এবং কেন দরকার ?

→ মেমরির সমস্যা | → কোয়ান্টাইজেশনের সমাধান

২

load_in_4bit প্যারামিটার

→ কী করে | → কীভাবে কাজ করে | → মেমরি সাশ্রয়ের হিসাব

৩

bnb_4bit_quant_type = 'nf4'

→ NF4 কী | → Normal Distribution কানেকশন | → ১৬টি Standard মান কীভাবে নির্ধারিত হয়

৪

৪-বিট কোয়ান্টাইজেশনের বিস্তারিত মেকানিজম

→ Weight রূপান্তর প্রক্রিয়া | → Matrix Shape | → Block-wise Quantization

৫

bnb_4bit_compute_dtype = torch.bfloat16

→ bfloat16 কী | → কেন ৪-বিটে হিসাব হয় না | → Dequantization প্রক্রিয়া

৬

৪-বিট থেকে ১৬-বিটে রূপান্তর (Dequantization)

→ কখন হয় | → সমীকরণ | → একই মান বারবার আসার প্রশ্ন

৭

Quantization Error ও তার সমাধান

→ তথ্যের ঘাটতি | → মডেলের সহনশীলতা | → Block-wise পদ্ধতি

৮

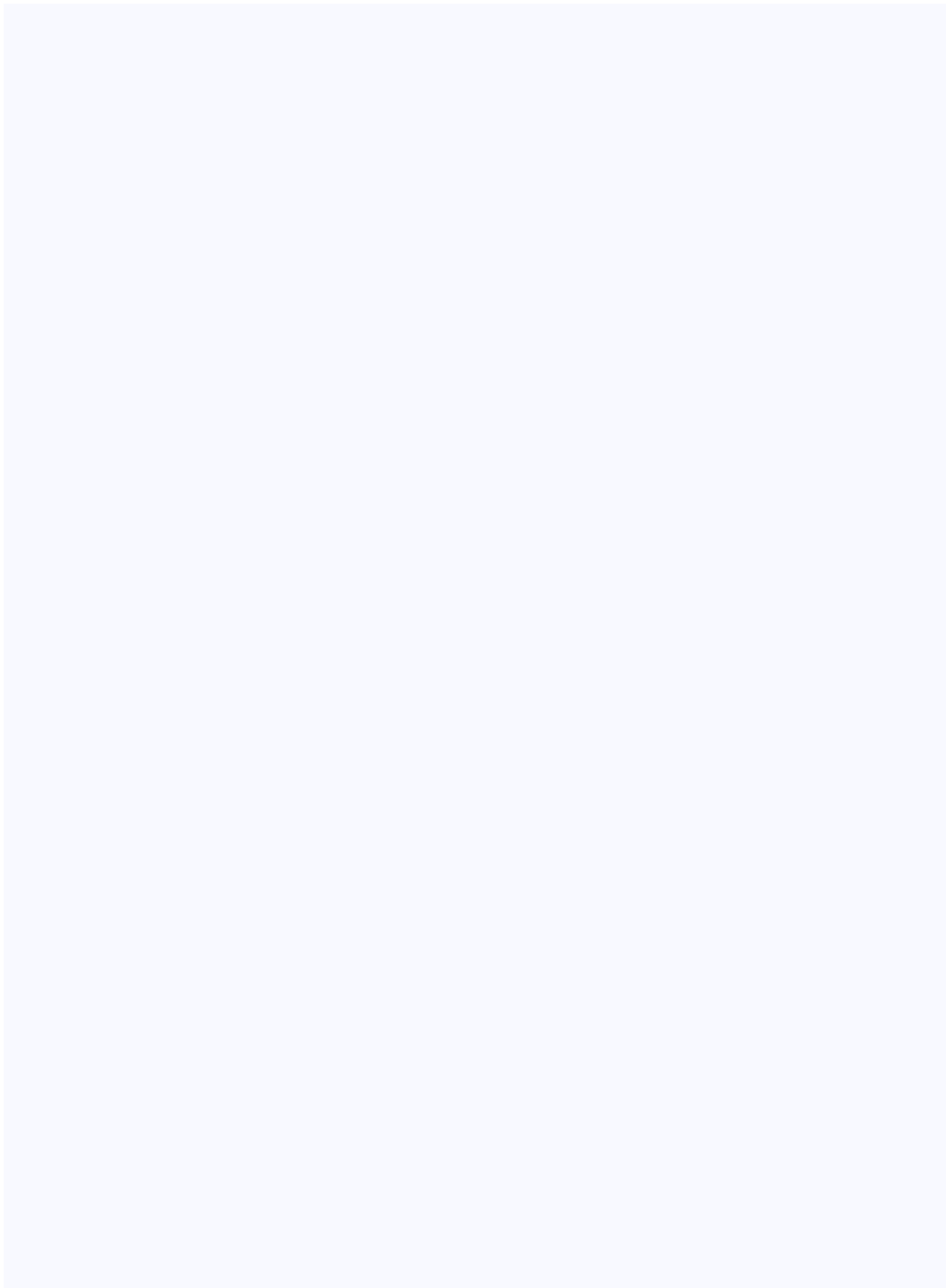
QLoRA ও LoRA Adapter

→ LoRA কী | → A ও B Matrix | → গাণিতিক সমীকরণ | → কেন ১৬-বিটে থাকে

৯

সম্পূর্ণ QLoRA কোড উদাহরণ

১০



BitsAndBytesConfig কী এবং কেন দরকার ?

বড় AI মডেলকে সাধারণ GPU-তে চালানোর সমস্যা ও সমাধান



মূল ধারণা

BitsAndBytesConfig হলো Hugging Face-এর **transformers** লাইব্রেরির একটি কনফিগারেশন ক্লাস, যা **bitsandbytes** লাইব্রেরির সাথে মিলে কাজ করে। এর মূল লক্ষ্য হলো বিশাল বড় Language Model (LLM)-কে সীমিত GPU মেমরিতে (VRAM) লোড করতে এবং ফাইন-টিউন করতে সাহায্য করা।



সমস্যাটি কী ?

একটি সাধারণ ৭ বিলিয়ন প্যারামিটারের মডেল (যেমন Llama-2-7B) float32 ফরম্যাটে রাখলে প্রায় **28 GB VRAM** লাগে! অধিকাংশ মানুষের কাছে এত শক্তিশালী GPU নেই।

মডেলের সাইজ	float32 (32-bit)	float16 (16-bit)	4-bit (NF4)
7B প্যারামিটার	~28 GB	~14 GB	~4 GB ✓
13B প্যারামিটার	~52 GB	~26 GB	~7 GB ✓
2B প্যারামিটার (Gemma)	~8 GB	~4 GB	~1.5 GB ✓

 উপমা — বুঝতে সাহায্য করবে

ধরুন আপনার কাছে একটি বিশাল বড় বই আছে (৫০০ পাতার)। আপনার ব্যাগে মাত্র ১০০ পাতার জায়গা। এখন আপনি যদি প্রতিটি পাতার মূল তথ্যটুকু ছোট ছোট কোডে লিখে রাখেন, তাহলে ব্যাগে সব ধরে যাবে! **BitsAndBytesConfig** ঠিক এই কাজটিই করে — মডেলের ওজনগুলোকে "কম্প্রেস" করে রাখে।

```
PYTHON

# প্রথমে প্রয়োজনীয় লাইব্রেরি ইনস্টল করুন
# pip install transformers bitsandbytes accelerate torch

from transformers import BitsAndBytesConfig, AutoModelForCausalLM
import torch

# BitsAndBytesConfig দিয়ে কনফিগারেশন তৈরি করা
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,          # 4-বিটে মেমরিতে লোড করো
    bnb_4bit_quant_type='nf4',  # NormalFloat4 ফরম্যাট ব্যবহার করো
    bnb_4bit_compute_dtype=torch.bfloat16 # হিসাবের সময় bfloat16
)

# মডেল লোড করা এই কনফিগ দিয়ে
model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-2b",
    quantization_config=bnb_config,
    device_map="auto"
)
```

load_in_4bit = True

মডেলের ওজনগুলোকে ৪-বিট ফরম্যাটে লোড করার নির্দেশ



এই প্যারামিটারটি কী করে ?

এটি মডেলকে নির্দেশ দেয় যে তার সব ওজন (weights) বা প্যারামিটারগুলো GPU মেমরিতে ৪-বিট ফরম্যাটে লোড করতে হবে। সাধারণত মডেলের ওজনগুলো ১৬-বিট বা ৩২-বিটে থাকে।

বিট এবং রেঞ্জের সম্পর্ক

ফরম্যাট	বিট সংখ্যা	মোট লেভেল	রেঞ্জ	উদাহরণ মান
float32	32 bit	4,294,967,296	বিশাল	0.12345678901
float16	16 bit	65,536	মাঝারি	0.1235
int8	8 bit	256	০ থেকে ২৫৫	128
NF4 (4-bit)	4 bit	16	Index: ০ থেকে ১৫	0111 → -0.276



গুরুত্বপূর্ণ পার্থক্য

৮-বিটে রেঞ্জ হয় ০ থেকে ২৫৫ (মোট ২৫৬টি মান)। ৪-বিটে মাত্র ১৬টি লেভেল ($2^4 = 16$), সেগুলো ইনডেক্স হিসেবে ব্যবহৃত হয়, সরাসরি ০ থেকে ১৫ সংখ্যা হিসেবে নয়।

মেমরি বাঁচানোর গাণিতিক হিসাব

উদাহরণ: একটি ৯x৯ ম্যাট্রিক্সে ৮১টি ওজন আছে।

float16-এ: $81 \times 16 \text{ bits} = 1,296 \text{ bits}$

4-bit-এ: $81 \times 4 \text{ bits} = 324 \text{ bits}$

সাশ্রয়: 75% মেমরি বাঁচে! (4x কম)



PYTHON

```
import torch
```

```
# মেমরি তুলনা দেখার কোড
```

```
matrix_float16 = torch.rand(9,9, dtype=torch.float16)
```

```
matrix_float32 = torch.rand(9,9, dtype=torch.float32)
```

```
print(f"float16 মেমরি: {matrix_float16.element_size() * matrix_float16.nelement()} bytes")
```

```
print(f"float32 মেমরি: {matrix_float32.element_size() * matrix_float32.nelement()} bytes")
```

```
# float16 মেমরি: 162 bytes (81 x 2 bytes)
```

```
# float32 মেমরি: 324 bytes (81 x 4 bytes)
```

```
# 4-bit হলে হতো: ~41 bytes (81 x 0.5 bytes)
```

মেমরিতে ওজন সংরক্ষণের তুলনা (৪টি ওজনের জন্য)

float32 — প্রতিটি ওজনে 32 bit (4 bytes)

00000

00000

00000

00000

00000

00000

00000

00000

4 টি ওজন $\times$ 4 bytes = 16 bytes

float16 – প্রতিটি ওজনে 16 bit (2 bytes)

0000

0000

0000

0000

4 টি ওজন $\times$ 2 bytes = 8 bytes

4-bit NF4 – প্রতিটি ওজনে মাত্র 4 bit (0.5 bytes)

0000

0001

4 টি ওজন $\times$ 0.5 bytes = 2 bytes ✓

bnb_4bit_quant_type = 'nf4'

NormalFloat4 — কীভাবে ১৬টি বিশেষ মান নির্ধারিত হয়?



NF4 (NormalFloat4) কী?

'nf4' এর পূর্ণরূপ হলো **NormalFloat4**। এটি ৪-বিট কোয়ান্টাইজেশনের একটি বিশেষ ধরন যা নিউরাল নেটওয়ার্কের ওজনগুলোর প্রকৃতি বিবেচনা করে ডিজাইন করা হয়েছে।



মূল অন্তর্দৃষ্টি

নিউরাল নেটওয়ার্কের ওজনগুলো সাধারণত **Normal Distribution** মেনে চলে — অর্থাৎ বেশিরভাগ ওজন শূন্যের কাছাকাছি থাকে, এবং খুব বড় বা খুব ছোট মান বিরল। NF4 এই বৈশিষ্ট্যকে কাজে লাগিয়ে সর্বোচ্চ নির্ভুলতা নিশ্চিত করে।

Normal Distribution এবং সমান এলাকা বিভাজন

NF4 তৈরির পদ্ধতি তিনটি ধাপে বোঝা যায়:

১

Normal Distribution চেনা

মডেলের ওজনগুলো -১ থেকে +১ এর মধ্যে একটি বেল কার্ভ (Bell Curve) আকারে ছড়িয়ে থাকে। শূন্যের কাছে সবচেয়ে বেশি ওজন জমাট বেঁধে থাকে।

২

সমান এলাকায় ভাগ করা (Equal Area Partitioning)

এই বেল কার্ভের মোট জায়গাকে (Area) সমান ১৬ ভাগে বিভক্ত করা হয়। এই কারণেই শূন্যের কাছাকাছি মানগুলো ঘনিষ্ঠভাবে বিন্যস্ত থাকে (কারণ সেখানে বেশি ওজন আছে), আর চরম মানগুলোর মধ্যে বেশি ব্যবধান থাকে।

প্রতিটি ভাগের মাঝখানের (Expected Value) মানটিকে সেই লেভেলের প্রতিনিধি হিসেবে নির্বাচন করা হয়। এই ১৬টি মান গাণিতিকভাবে গণনা করে আগে থেকেই নির্ধারণ করে রাখা হয়।

NF4-এর ১৬টি Standard মান (Lookup Table)

Index 0 0000 -1.0000	Index 1 0001 -0.6962	Index 2 0010 -0.5251	Index 3 0011 -0.3949
Index 4 0100 -0.2849	Index 5 0101 -0.1847	Index 6 0110 -0.0911	Index 7 0111 0.0000
Index 8 1000 +0.0796	Index 9 1001 +0.1609	Index 10 1010 +0.2461	Index 11 1011 +0.3379
Index 12 1100 +0.4407	Index 13 1101 +0.5626	Index 14 1110 +0.7230	Index 15 1111 +1.0000



লক্ষ্য করুন

শূন্যের (0) কাছের মানগুলো (Index 6, 7, 8) একে অপরের অনেক কাছাকাছি (-0.09, 0.00, +0.08)। কিন্তু চরম মানগুলোর (Index 0, 1) মধ্যে ব্যবধান বেশি (-1.00, -0.70)। এটাই NF4-এর চালাকি!

কেন সরাসরি ০ থেকে ১৫ ব্যবহার করা হয় না?

✗ সাধারণ পদ্ধতি (UNIFORM)

সমান দূরত্বে মান রাখলে শূন্যের কাছে পর্যাপ্ত অপশন থাকে না। বেশিরভাগ ওজন এক জায়গায় "ভিড়" করবে।

✓ NF4 পদ্ধতি

ঘনত্ব অনুযায়ী মান রাখা হয়। শূন্যের কাছে বেশি অপশন, দূরে কম। ফলে তথ্যের ক্ষতি সর্বনিম্ন।

```

import numpy as np

# NF4 এর ১৬টি স্ট্যান্ডার্ড মান (প্রি-ডিফাইনড Lookup Table)
NF4_LOOKUP = [
    -1.0000, -0.6962, -0.5251, -0.3949,
    -0.2849, -0.1847, -0.0911, 0.0000,
    0.0796, 0.1609, 0.2461, 0.3379,
    0.4407, 0.5626, 0.7230, 1.0000
]

def quantize_to_nf4(weight):
    """একটি ওজনকে NF4 ইনডেক্সে রূপান্তর করে"""
    # ওজনের absmax দিয়ে Normalize করো (-1 থেকে 1 এর মধ্যে আনো)
    absmax = np.abs(weight).max()
    normalized = weight / (absmax + 1e-8)

    # সবচেয়ে কাছের NF4 মানের ইনডেক্স খুঁজে বের করো
    indices = np.argmin(
        np.abs(np.array(NF4_LOOKUP)[np.newaxis, :] - normalized),
        axis=1
    )
    return indices, absmax # ইনডেক্স এবং scale factor ফেরত দাও

def dequantize_from_nf4(indices, absmax):
    """NF4 ইনডেক্স থেকে ওজন পুনরুদ্ধার করো"""
    nf4_vals = np.array(NF4_LOOKUP)[indices]
    return nf4_vals * absmax # Scale back

# উদাহরণ: একটি ছোট ওজন ভেক্টর
original_weights = np.array([0.12, -0.45, 0.78, -0.09, 0.33])
indices, scale = quantize_to_nf4(original_weights)
recovered = dequantize_from_nf4(indices, scale)

print(f"মূল ওজন: {original_weights}")
print(f"NF4 ইনডেক্স: {indices}")

```

```
print(f"পুনরুদ্ধার: {recovered.round(4)}")  
print(f"ত্রুটি: {np.abs(original_weights - recovered).mean():.4f}")
```

৪-বিট কোয়ান্টাইজেশনের বিস্তারিত মেকানিজম

মডেল লোড হওয়ার সময় Weight-গুলোর কী হয় ?



Weight রূপান্তর প্রক্রিয়া

যখন আপনি `load_in_4bit=True` দিয়ে মডেল লোড করেন, তখন নিচের ধাপগুলো ঘটে:

১

ব্লক ভাগ করা (Block-wise Partitioning)

মডেলের প্রতিটি ওজন ম্যাট্রিক্সকে ছোট ছোট ব্লকে ভাগ করা হয় (সাধারণত প্রতি ৬৪টি ওজন একটি ব্লক)। প্রতিটি ব্লকের জন্য আলাদা Scale Factor (absmax) রাখা হয়।

২

Scaling (নরমালাইজেশন)

প্রতিটি ব্লকের সর্বোচ্চ পরম মান (absolute maximum) দিয়ে সব ওজনকে ভাগ করা হয়। এতে মানগুলো -১ থেকে +১ এর মধ্যে চলে আসে।

৩

Mapping (কাছের NF4 মান খোঁজা)

প্রতিটি Normalized ওজনের জন্য NF4 Lookup Table থেকে সবচেয়ে কাছের মানের ইনডেক্স (০-১৫) বেছে নেওয়া হয়।

৪

মেমরিতে সংরক্ষণ

মেমরিতে শুধু ৪-বিটের ইনডেক্স এবং প্রতিটি ব্লকের ১৬-বিটের Scale Factor সেভ করা হয়। মূল ১৬-বিট ওজনগুলো মুছে যায়।

Matrix Shape পরিবর্তন হয় না!



গুরুত্বপূর্ণ তথ্য

কোয়ান্টাইজেশনের পর ম্যাট্রিক্সের আকার (Shape) একদম পরিবর্তন হয় না। একটি ৯×৯ ম্যাট্রিক্স ৯×৯ ই থাকে। শুধু প্রতিটি ঘরের মানের নির্ভুলতা কমে যায় — অনেকগুলো ঘরে এখন একই মান থাকতে পারে।

```
PYTHON – Block-wise Quantization বিস্তারিত

import numpy as np

NF4_LOOKUP = [
    -1.0000, -0.6962, -0.5251, -0.3949,
    -0.2849, -0.1847, -0.0911, 0.0000,
    0.0796, 0.1609, 0.2461, 0.3379,
    0.4407, 0.5626, 0.7230, 1.0000
]

def blockwise_quantize(matrix, block_size=4):
    """
    একটি ম্যাট্রিক্সকে Block-wise NF4 কোয়ান্টাইজেশন করো।
    বাস্তবে block_size=64, এখানে সহজবোধ্যতার জন্য 4 ব্যবহার করা হলো।
    """
    flat = matrix.flatten() # ম্যাট্রিক্সকে 1D করো
    n = len(flat)
    quantized_indices = np.zeros(n, dtype=np.uint8)
    scale_factors = []

    for i in range(0, n, block_size):
        block = flat[i:i+block_size]
        absmax = np.abs(block).max() + 1e-8
        scale_factors.append(absmax)

    # প্রতিটি ব্লকের মানগুলো নরমালাইজ করো
    normalized = block / absmax
```

```

# NF4 Lookup Table থেকে কাছের মানের ইনডেক্স নাও
for j, val in enumerate(normalized):
    idx = np.argmin(np.abs(np.array(NF4_LOOKUP) - val))
    quantized_indices[i+j] = idx

return quantized_indices.reshape(matrix.shape), np.array(scales)

# উদাহরণ: একটি 4x4 ম্যাট্রিক্স
weight_matrix = np.random.randn(4, 4).astype(np.float32)
quantized, scales = blockwise_quantize(weight_matrix, block_size=2)

print("মূল ম্যাট্রিক্স (float32):")
print(weight_matrix.round(4))
print("\nকোয়ান্টাইজড (NF4 ইনডেক্স, 0-15):")
print(quantized)
print("\nShape পরিবর্তন হয়নি:", weight_matrix.shape, "→", quantized.shape)
print("\nমেমরি সাশ্রয়:", weight_matrix.nbytes, "bytes →", quantized.nbytes, "bytes")

```

🎯 উপমা — ৮১টি ওজন ও ১৬টি লেভেলের সমস্যা

আপনার কাছে ৮১ রকমের আলাদা শেডের রঙের পেন্সিল আছে (যেমন: ৮১ রকম লালের ছায়া)।

কিন্তু আপনার কাছে মাত্র ১৬টি রঙ আঁটে। তখন আপনি কাছাকাছি সব লাল শেডকে একটি

"প্রতিনিধি লাল" দিয়ে প্রতিস্থাপন করেন। ছবিটা ১০০% নিখুঁত না হলেও, যা আঁকা সেটা বোঝা যায়

— গোলাপ আঁকলে গোলাপই মনে হয়!

bnb_4bit_compute_dtype = torch.bfloat16

হিসাবের সময় কোন Data Type ব্যবহার হবে ?



bfloat16 কী ?

bfloat16 এর পূর্ণরূপ হলো **Brain Floating Point 16-bit**। গুগলের AI রিসার্চ টিম এটি তৈরি করেছিল। এটি কোনো Matrix Shape নয় — এটি একটি **Data Type**।

Data Type	মোট বিট	Sign বিট	Exponent বিট	Mantissa বিট	বৈশিষ্ট্য
float32	32	1	8	23	সর্বোচ্চ নির্ভুলতা
float16	16	1	5	10	সীমিত রেঞ্জ, Overflow ঝুঁকি
bfloat16	16	1	8	7	float32-এর রেঞ্জ, কিন্তু ১৬-বিট! ✓



bfloat16-এর চালাকি

bfloat16-এ **Exponent** বিট ৮টি — ঠিক float32-এর মতো। তাই এটি float32-এর মতো বিশাল বড় সংখ্যাও ধারণ করতে পারে (Overflow হয় না), অথচ জায়গা নেয় মাত্র ১৬ বিট। Mantissa (সূক্ষ্মতা) কিছুটা কমে, কিন্তু LLM-এর জন্য তা গ্রহণযোগ্য।

কেন সরাসরি ৪-বিটে হিসাব করা যায় না?



Data Type Mismatch

আপনার ইনপুট (প্রম্পট টেক্সট) প্রসেস হয়ে **bfloat16** দশমিক সংখ্যায় পরিণত হয়। কিন্তু ওজনগুলো আছে ৪-বিট ইনডেক্সে (০-১৫)। GPU এই দুই ভিন্ন ধরনের সংখ্যা সরাসরি গুণ করতে পারে না।



ইনডেক্স দিয়ে গুণ করলে ভুল হয়

Index ৩ এর আসল মান -০.৩৯৪৯। কিন্তু আপনি যদি সরাসরি ৩ দিয়ে গুণ করেন, ফলাফল সম্পূর্ণ ভুল হবে। তাই আগে ইনডেক্স → আসল মানে রূপান্তর (Dequantization) করতে হবে।



PYTHON – bfloat16 বনাম float16 Overflow পরীক্ষা

```
import torch

# Overflow পরীক্ষা
large_val = torch.tensor(70000.0)

f16 = large_val.to(torch.float16)
bf16 = large_val.to(torch.bfloat16)
f32 = large_val.to(torch.float32)

print(f"মূল মান: {large_val.item()}")
print(f"float16: {f16.item()}") # → inf (Overflow হয়!)
print(f"bfloat16: {bf16.item()}") # → 70016.0 (ঠিকঠাক!)
print(f"float32: {f32.item()}") # → 70000.0
```

```
# কেন compute_dtype bfloat16 দরকার তার প্রদর্শনী
# 8-বিট ইনডেক্স দিয়ে সরাসরি গুণ করলে কী হয়?

input_val = torch.tensor(0.5, dtype=torch.bfloat16)
index_val = 3          # NF4 ইনডেক্স
actual_nf4_val = -0.3949 # NF4 lookup থেকে প্রকৃত মান

wrong_result = float(input_val) * index_val
correct_result = float(input_val) * actual_nf4_val

print(f"\nইনপুট: {float(input_val)}, NF4 ইনডেক্স: {index_val}")
print(f"ভুল গুণফল (ইনডেক্স দিয়ে): {wrong_result}") # 1.5 — সম্পূর্ণ ভুল!
print(f"সঠিক গুণফল (NF4 মান দিয়ে): {correct_result:.4f}") # -0.1975 — সঠিক
```

৪-বিট থেকে ১৬-বিটে রূপান্তর (Dequantization)

হিসাব করার সময় "On-the-fly" রূপান্তর — কখন, কীভাবে এবং সমীকরণ

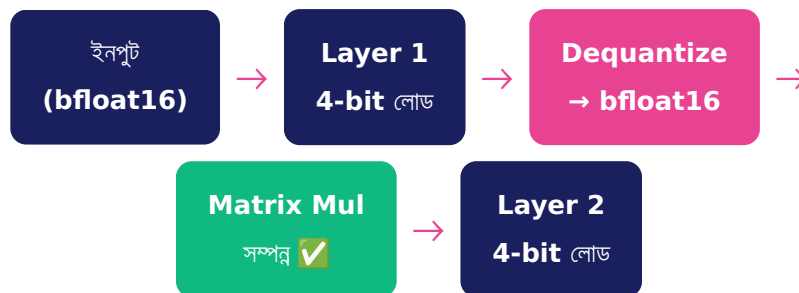


Dequantization প্রক্রিয়া



পুরো মডেল একসাথে রূপান্তর হয় না!

যদি পুরো মডেল একসাথে ১৬-বিটে রূপান্তর করা হতো, তাহলে VRAM আবার ভরে যেত! বরং এটি হয় "On-the-fly" — প্রয়োজনের সময়, একটি লেয়ার একটু একটু করে।



Dequantization সমীকরণ

৪-বিট ইনডেক্স থেকে আসল ওজন পুনরুদ্ধারের সূত্র:

$$\text{Weight_bfloat16} = \text{Scale} \times \text{NF4_Value}(4\text{-bit_Index})$$

যেখানে: **NF4_Value** = Lookup Table থেকে সংশ্লিষ্ট দশমিক মান
Scale = প্রতিটি ব্লকের জন্য সেভ রাখা **absmax** মান (১৬-বিট)

উদাহরণ: ধরুন ইনডেক্স = ৩, Scale = ০.৮

NF4_Value(3) = -0.3949

Weight = 0.8 × (-0.3949) = -0.3159

রূপান্তরের পরও একই মান থাকে

এই প্রশ্নটি অত্যন্ত গুরুত্বপূর্ণ: যেসব ওজনের ৪-বিট ইনডেক্স একই ছিল, তাদের ১৬-বিট মানও একই হবে।
হারানো তথ্য ফিরে আসে না।

```
PYTHON — একই মান থাকার প্রদর্শনী

import numpy as np

NF4_LOOKUP = [-1.0, -0.6962, -0.5251, -0.3949, -0.2849, -0.1847,
               -0.0911, 0.0, 0.0796, 0.1609, 0.2461, 0.3379,
               0.4407, 0.5626, 0.7230, 1.0]

# ৫টি কাছাকাছি ওজন যারা একই NF4 ইনডেক্সে পড়বে
similar_weights = np.array([0.11, 0.12, 0.13, 0.14, 0.15])
absmax = similar_weights.max()

# কোয়ান্টাইজ করো
normalized = similar_weights / absmax
indices = [np.argmin(np.abs(np.array(NF4_LOOKUP) - v)) for v in normalized]

# Dequantize করো (১৬-বিটে রূপান্তর)
recovered = [NF4_LOOKUP[i] * absmax for i in indices]

print("মূল ওজন → NF4 ইনডেক্স → পুনরুদ্ধার:")
```

```
for orig, idx, rec in zip(similar_weights, indices, recovered):  
    print(f" {orig:.2f} → Index {idx:2d} → {rec:.4f}")
```

আউটপুট দেখাবে: কাছাকাছি মানগুলো একই ইনডেক্সে পড়ে, একই মান ফিরে আসে

🎯 উপমা — জিপ ফাইলের মতো

মডেলের ওজনগুলো মেমরিতে ৪-বিট "ZIP ফাইলের" মতো সংরক্ষিত থাকে। গান প্লে করতে হলে ZIP খুলতে হয় (Dequantize)। ZIP খোলার পর গানটি বাজানো হয় (Matrix Multiplication)। বাজানো শেষে সেই unzipped কপি মুছে যায় — মেমরিতে আবার ZIP ফাইলই থাকে।

Quantization Error ও তার সমাধান

তথ্যের ঘাটতি, মডেলের সহনশীলতা ও Block-wise পদ্ধতি



Quantization Error কী ?

যখন ৮১টি ভিন্ন ওজনকে মাত্র ১৬টি মানে ফেলে দেওয়া হয়, তখন কিছু তথ্য অনিবার্যভাবে হারিয়ে যায়। এটাই Quantization Error।

কেন মডেল তারপরও কাজ করে ?

১

মডেলের সহনশীলতা (**Robustness**)

নিউরাল নেটওয়ার্কে সবচেয়ে গুরুত্বপূর্ণ হলো ওজনগুলোর সামগ্রিক দিক বা প্যাটার্ন, নিখুঁত দশমিক মান নয়। কাছাকাছি মান দিয়ে প্রতিস্থাপন করলেও মডেলের মূল "বোঝার ক্ষমতা" অটুট থাকে।

২

Block-wise Quantization

প্রতি ৬৪টি ওজনের জন্য আলাদা Scale Factor ব্যবহার করা হয়। এতে বিভিন্ন ব্লকে ওই ১৬টি লেভেলের রেঞ্জ আলাদা হয়, ফলে পুরো মডেলজুড়ে আরও বেশি বৈচিত্র্য রক্ষা পায়।

৩

LoRA Adapter (QLoRA-এর মূল কৌশল)

৪-বিটে কমে যাওয়া ঘাটতিগুলো পূরণ করতে ছোট ছোট ১৬-বিটের Adapter যোগ করা হয় — যা Fine-tuning-এর সময় নতুন করে শেখে।



PYTHON – Quantization Error পরিমাপ

```
import numpy as np
```

```

NF4_LOOKUP = [-1.0, -0.6962, -0.5251, -0.3949, -0.2849,
               -0.1847, -0.0911, 0.0, 0.0796, 0.1609,
               0.2461, 0.3379, 0.4407, 0.5626, 0.7230, 1.0]

def compute_quantization_error(original):
    """Quantization Error গণনা করো"""
    absmax = np.abs(original).max() + 1e-8
    norm = original / absmax
    indices = np.argmin(
        np.abs(np.array(NF4_LOOKUP)[np.newaxis, :] - norm[:, np.newaxis])
    )
    recovered = np.array(NF4_LOOKUP)[indices] * absmax

    mae = np.abs(original - recovered).mean()
    mse = ((original - recovered) ** 2).mean()
    return mae, mse

# বিভিন্ন ধরনের ওজনে Error পরিমাপ
np.random.seed(42)

# সাধারণ নরমাল ডিস্ট্রিবিউশন ওজন (যেমন আসল মডেলে থাকে)
normal_weights = np.random.randn(1000) * 0.5
mae_n, mse_n = compute_quantization_error(normal_weights)

print(f"নরমাল ডিস্ট্রিবিউশন ওজনে:")
print(f"  MAE (গড় পরম ত্রুটি): {mae_n:.6f}")
print(f"  MSE (গড় বর্গ ত্রুটি): {mse_n:.6f}")
print(f"  ত্রুটির মাত্রা: অত্যন্ত ছোট ✅")

```

QLoRA ও LoRA Adapter

৪-বিটের ঘাটতি পূরণ করার জাদুকরী কৌশল



LoRA (Low-Rank Adaptation) কী?

LoRA হলো একটি Fine-tuning পদ্ধতি যেখানে মূল মডেলের ওজন পরিবর্তন না করে, তার সাথে দুটি ছোট **Matrix (A ও B)** যোগ করা হয়। এই দুটি Matrix-ই Fine-tuning-এর সময় শেখে।

LoRA-র গাণিতিক সমীকরণ

$$\text{Output} = (W_{\text{base}} \times \text{Input}) + (A \times B \times \text{Input})$$

W_base = মূল মডেলের ৪-বিট ওজন (Frozen, পরিবর্তন হয় না)

A ও B = নতুন ছোট Matrix (১৬-বিট bfloat16, শুধু এগুলোই শেখে)

r = Rank (A ও B-এর মাঝের ডাইমেনশন, সাধারণত ৮-৬৪)

কেন **A ও B Matrix** ১৬-বিটে থাকে?



সূক্ষ্মতার প্রয়োজন

Fine-tuning-এর সময় Gradient (ঢাল) অনুযায়ী ওজন আপডেট করতে হয়। এই পরিবর্তনগুলো অত্যন্ত সূক্ষ্ম। ৪-বিটে মাত্র ১৬টি মান থাকায় এই সূক্ষ্ম আপডেট সম্ভব নয়। তাই A ও B কে ১৬-বিটে রাখা হয়।



ছোট হওয়ায় মেমরি সমস্যা নেই

মূল মডেলে যদি কোটি কোটি প্যারামিটার থাকে, LoRA Adapter-এ থাকে তার মাত্র ১% বা তারও কম। এই সামান্য কিছু ওজন ১৬-বিটে রাখলেও মেমরিতে বিশেষ চাপ পড়ে না।

🕒 উপমা — ঝাপসা চশমা ও অতিরিক্ত লেন্স

ভাবুন আপনি একটি ঝাপসা চশমা (৪-বিট মডেল) পরে আছেন — দেখতে পাচ্ছেন, কিন্তু পরিষ্কার নয়। এখন আপনি তার ওপর একটি অতিরিক্ত ছোট পরিষ্কার লেন্স (LoRA Adapter) পরে নিলেন। পুরো দৃশ্য আবার পরিষ্কার হয়ে গেল! মূল চশমা পরিবর্তন না করেই সমস্যা সমাধান।



PYTHON — LoRA গণিতের প্রদর্শনী

```
import torch
import torch.nn as nn

class LoRALayer(nn.Module):
    """QLoRA-এর মতো একটি LoRA লেয়ার — ধারণা বোঝার জন্য"""
    def __init__(self, in_features, out_features, rank=4):
        super().__init__()

        # মূল ওজন: বাস্তবে এটি ৪-বিটে থাকত (Frozen)
        self.base_weight = nn.Parameter(
            torch.randn(out_features, in_features),
            requires_grad=False # ফ্রোজেন — শেখে না
        )

        # LoRA Matrix A (bfloat16, শেখে)
        self.lora_A = nn.Parameter(
            torch.randn(rank, in_features, dtype=torch.bfloat16)
        )

        # LoRA Matrix B (bfloat16, শেখে, শূন্য দিয়ে শুরু)
        self.lora_B = nn.Parameter(
            torch.zeros(out_features, rank, dtype=torch.bfloat16)
        )

        self.scaling = 1.0 / rank
```

```

def forward(self, x):
    # মূল ওজন দিয়ে হিসাব (বাস্তবে dequantize হয়ে)
    base_out = nn.functional.linear(x.float(), self.base_weight)

    # LoRA অ্যাডাপ্টার:  $B \times A \times x$ 
    lora_out = nn.functional.linear(
        nn.functional.linear(x.to(torch.bfloat16), self.lora_A),
        self.lora_B
    ) * self.scaling

    # দুটো যোগ করো:  $Output = W_{base} \times x + B \times A \times x$ 
    return base_out + lora_out.float()

# ব্যবহার
layer = LoRALayer(in_features=8, out_features=8, rank=4)
x = torch.randn(1, 8)
output = layer(x)

# কোন প্যারামিটার শিখছে?
for name, param in layer.named_parameters():
    print(f"{name}: শিখছে={param.requires_grad}, dtype={param.dtype}")

```

```
import torch

from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainingArguments
)

from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer

from datasets import load_dataset

# =====
# ধাপ ১: BitsAndBytesConfig তৈরি করা
# =====

bnb_config = BitsAndBytesConfig(
```

```

load_in_4bit=True,
# 4-বিট ফরম্যাটে মেমরিতে লোড করো

bnb_4bit_quant_type='nf4',
# NormalFloat4 ব্যবহার করো (Normal Distribution-এর জন্য অপ্টিমাইজড)

bnb_4bit_compute_dtype=torch.bfloat16,
# হিসাবের সময় bfloat16 ব্যবহার করো (Overflow এড়াতে)

bnb_4bit_use_double_quant=True,
# Double Quantization: Scale Factor-ও কনস্টাইজ করো (আরও মেমরি সাশ্রয়)
)

# =====
# ধাপ ২: মডেল লোড করা
# =====

model_name = "google/gemma-2b"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config, # আমাদের 4-bit কনফিগ
    device_map="auto",             # GPU/CPU স্বয়ংক্রিয়ভাবে নির্বাচন
    trust_remote_code=True
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

print(f"মডেল লোড হয়েছে! VRAM ব্যবহার: {torch.cuda.memory_allocated()}")

# =====
# ধাপ 3: LoRA কনফিগ তৈরি করা
# =====

lora_config = LoraConfig(
    r=16, # Rank: A ও B Matrix-এর মাঝের ডাইমেনশন
    # বড় r = বেশি পরামিটার, বেশি ক্ষমতা, বেশি মেমরি

    lora_alpha=32, # Scaling factor (সাধারণত  $r \times 2$ )
    # Output = base + (lora_alpha/r) × LoRA_output

```

```

target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
# কোন কোন Layer-এ LoRA Adapter যোগ করবে?

lora_dropout=0.05, # Overfitting এড়াতে Dropout

bias="none",      # Bias ফ্রোজেন রাখো

task_type=TaskType.CAUSAL_LM # টাস্কের ধরন
)

# মডেলে LoRA Adapter যোগ করো
model = get_peft_model(model, lora_config)

# কতটা পরামিটার শিখছে?
trainable, total = 0, 0
for p in model.parameters():
    total += p.numel()
    if p.requires_grad: trainable += p.numel()
print(f"মোট পরামিটার: {total:,}")
print(f"Trainable (LoRA): {trainable:,} ({100 * trainable/total}% of total)")

# =====
# ধাপ ৪: Training Arguments ও Trainer
# =====

training_args = TrainingArguments(
    output_dir="./qlora_output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    bf16=True,      # bfloat16-এ ট্রেনিং
    logging_steps=100,
    save_steps=500,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
)

# ডেটাসেট লোড (উদাহরণ)

```

```
dataset = load_dataset("tatsu-lab/alpaca", split="train[:100000]")

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    args=training_args,
    max_seq_length=512,
)

# ট্রেনিং শুরু করো!
trainer.train()

# মডেল সেভ করো
model.save_pretrained("./qlora_adapter")
```

সারসংক্ষেপ ও তুলনামূলক পর্যালোচনা

সব কিছু একনজরে — QLoRA-র সম্পূর্ণ চিত্র



প্রতিটি প্যারামিটারের ভূমিকা একনজরে

প্যারামিটার	মান	কাজ	না দিলে কী হতো
<code>load_in_4bit</code>	True	ওজন ৪-বিটে মেমরিতে সংরক্ষণ, মেমরি ৭৫% কমায়	মডেল লোড হতো না, GPU VRAM শেষ হয়ে যেত
<code>bnb_4bit_quant_type</code>	'nf4'	Normal Distribution অনুযায়ী অপ্টিমাইজড ১৬- মানের স্কিম	সাধারণ int4 ব্যবহার হতো, নির্ভুলতা কম হতো
<code>bnb_4bit_compute_dtype</code>	bfloat16	হিসাবের সময় ৪- বিট → ১৬-বিটে সাময়িক রূপান্তর	Type Mismatch Error অথবা ভুল আউটপুট আসত

QLoRA-র সম্পূর্ণ প্রবাহ চিত্র



🎯 মূল শিক্ষা — সহজ ভাষায়



স্টোরেজ: ওজনগুলো NF4 ফরম্যাটে ৪-বিটের "সংকুচিত প্যাকেজে" থাকে। প্রতিটি ঘরে ০ থেকে ১৫ এর মধ্যে একটি ইনডেক্স থাকে — NF4 Lookup Table-এর পয়েন্টর।



গণনা: হিসাবের সময় ইনডেক্স → আসল দশমিক মান (bfloat16) রূপান্তর হয়, গুণ হয়, তারপর ফলাফল ফেরত দেওয়া হয়। মেমরিতে ৪-বিটই থাকে।



ঘাটতি পূরণ: ৪-বিটে হারানো তথ্য LoRA Adapter (A ও B Matrix, bfloat16) পূরণ করে। Fine-tuning-এ শুধু এই Adapter-ই শেখে — মূল মডেল ফ্রোজেন থাকে।



ফলাফল: ১৬-বিট মডেলের ~৯৯% পারফরম্যান্স মাত্র ১/৪ মেমরিতে। সাধারণ GPU-তেও বিশাল LLM Fine-tune করা সম্ভব হয়।

দ্রুত রেফারেন্স কার্ড

```
PYTHON — সম্পূর্ণ রেফারেন্স

from transformers import BitsAndBytesConfig
import torch

#####

BitsAndBytesConfig প্যারামিটার সারসংক্ষেপ:
```



```
load_in_4bit = True
```

- মেমরিতে 8-বিট NF4 ইনডেক্স হিসেবে সংরক্ষণ
- মেমরি ৭৫% কমায় (4X সাশ্রয়)
- Shape পরিবর্তন হয় না, কিন্তু মান-বৈচিত্র্য কমে

```
bnb_4bit_quant_type = 'nf4'
```

- NormalFloat4: Normal Distribution- অপটিমাইজড
- ১৬টি পি-ডিফাইনড মান (Lookup Table)
- শূন্যের কাছে বেশি রেজোলিউশন, দূরে কম
- বিকল্প: 'fp4' (সাধারণ 4-bit float)

```
bnb_4bit_compute_dtype = torch.bfloat16
```

- গণনার সময় 4-bit → bfloat16 রূপান্তর
- bfloat16: float32-এর রেঞ্জ + 16-bit এর দক্ষতা
- Overflow এড়ায় (float16-এর বিপরীতে)

```
bnb_4bit_use_double_quant = True (অতিরিক্ত অপশন)
```

- Scale Factor-ও কোয়ান্টাইজ করে
- আরও ~0.4 bit/parameter সাশ্রয়

```
''''''
```

```
bnb_config = BitsAndBytesConfig(
```

```
    load_in_4bit=True,
```

```
    bnb_4bit_quant_type='nf4',
```

```
    bnb_4bit_compute_dtype=torch.bfloat16,
```

```
    bnb_4bit_use_double_quant=True # বোনাস: আরও মেমরি সাশ্রয়
```

```
)
```



অভিনন্দন!

আপনি এখন BitsAndBytesConfig-এর প্রতিটি প্যারামিটার, NF4 কোয়ান্টাইজেশনের গণিত, Dequantization প্রক্রিয়া, এবং QLoRA-র সম্পূর্ণ কার্যপদ্ধতি বিস্তারিত বুঝে ফেলেছেন। এই জ্ঞান ব্যবহার করে আপনি এখন সীমিত GPU-তে যেকোনো বড় LLM ফাইন-টিউন করতে পারবেন!



BitsAndBytesConfig টেকনিক্যাল গাইড | QLoRA ও ৪-বিট কোয়ান্টাইজেশন বাংলা রেফারেন্স

তথ্যসূত্র: Dettmers et al. (QLoRA, 2023) | Hugging Face transformers | bitsandbytes লাইব্রেরি